Solution: Dot Product of Two Sparse Vectors

Problem Statement

Given two `sparse vectors`, efficiently compute the `dot product` of them.

A `sparse vector` is one in which most elements are zero.

Implement class `Solution`:

- `Solution(nums)` Initializes the object with the vector nums
- `dotProduct(vec)` Calculates the dot product between the `vec` and instance of `Solution`.

Examples

- **Example 1**:
  - **Input**: vec1 = `[1, 0, 0, 2]`, vec2 = `[2, 3, 0, 1]`
  - **Expected Output**: `4`
  - **Justification**: The dot product is `(1*2) + (0*3) + (0*0) + (2*1) = 2 + 0 + 0 + 2 = 4`.
- **Example 2**:
  - **Input**: vec1 = `[0, 0, 0, 0]`, vec2 = `[1, 1, 1, 1]`
  - **Expected Output**: `0`
  - **Justification**: Since `vec1` is all zeros, the dot product is `0` regardless of `vec2`'s values.
- **Example 3**:
  - **Input**: vec1 = `[1, 0, 0, 2, 3]`, vec2 = `[0, 3, 0, 4, 0]`
  - **Expected Output**: `8`
  - **Justification**: The dot product is `(1*0) + (0*3) + (0*0) + (2*4) + (3*0) = 8`.

Solution

To solve this problem, we focus on the sparse nature of the vectors. A sparse vector has many zeros, which contribute nothing to the dot product. Thus, we avoid computations involving these zeros. The effective approach is to iterate through the non-zero elements of the vectors and compute their products only for corresponding indices. This is efficient because it significantly reduces the number of multiplications and additions compared to a naive approach that processes every element.

The algorithm leverages a data structure like a hash map to store the non-zero elements of one vector, indexed by their positions. We then iterate through the second vector, and for each non-zero element, we check if there's a corresponding non-zero element in the first vector (using the hash map). If so, we multiply these elements and add the result to a running total, which eventually becomes the dot product.

Step-by-step Algorithm

1. **Implement the Constructor** (`Solution()`):
   - The constructor of the `Solution` class takes an array of integers as input, representing a vector.
   - Inside the constructor, iterate through the input array.
   - For each element, check if it is non-zero.
   - If an element is non-zero, store its value and index in the data structure (such as a HashMap or Dictionary).

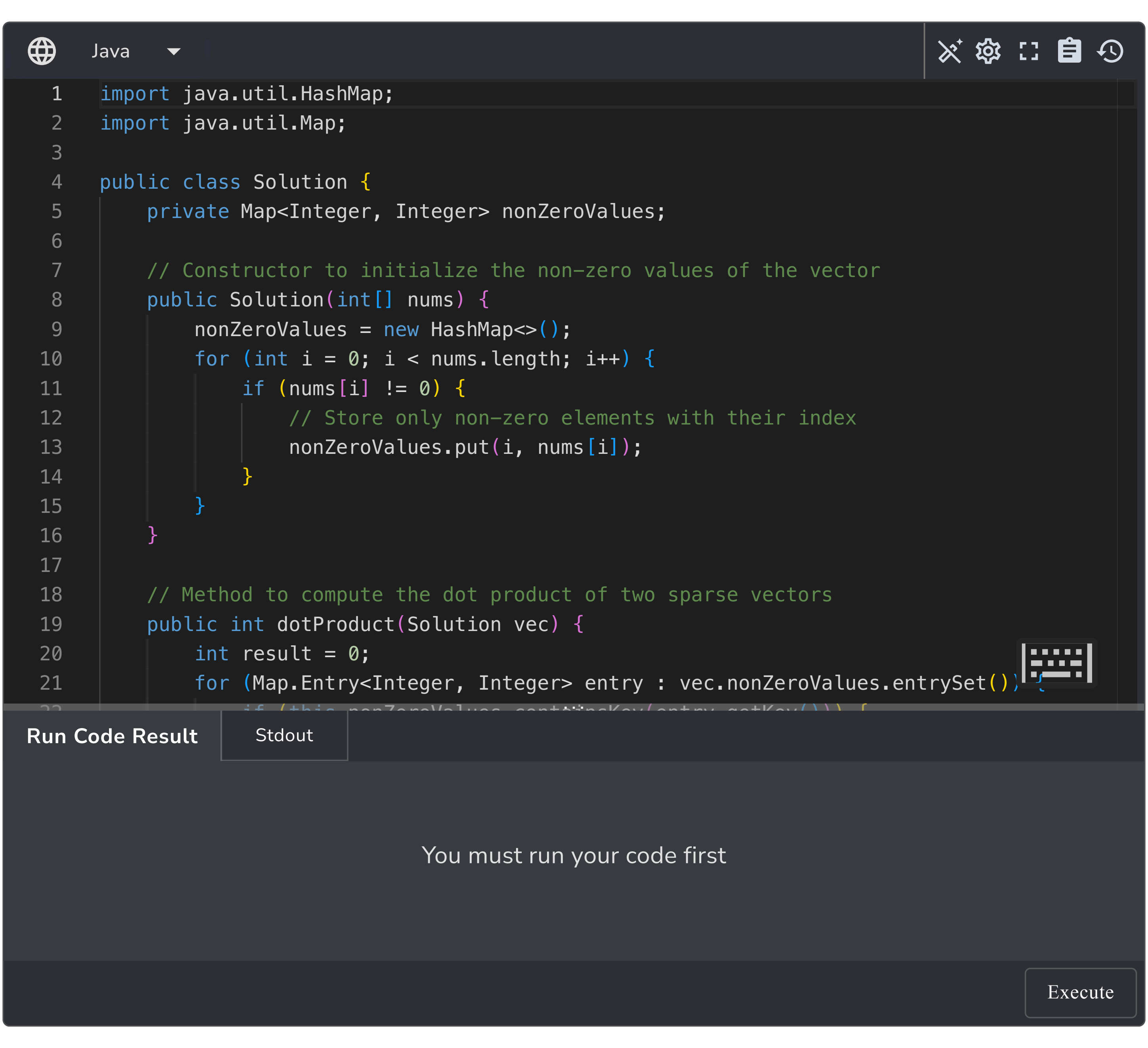
2. **Implement the** `dotProduct` **Method**:
   - This method calculates the dot product of the current vector instance with another vector.
   - It takes another `Solution` class instance as input.
   - Initialize a variable, say `result`, to 0. This will hold the result of the dot product.
   - Iterate through the non-zero elements of the other `Solution` instance.
   - For each non-zero element in the other vector, check if the current vector has a non-zero element at the same index.
   - If it does, multiply the two non-zero elements and add the product to `result`.
   - Return `result` as the dot product of the two vectors.

Algorithm Walkthrough

Example Input: `vec1 = [1, 0, 0, 2, 3]`, `vec2 = [0, 3, 0, 4, 0]`

1. Create two `Solution` objects, `solutionVec1` and `solutionVec2`, by passing `vec1` and `vec2` to the constructors respectively.
2. Inside `Solution()` for `solutionVec1`, the constructor stores the non-zero elements and their indices in a data structure: `{0: 1, 3: 2, 4: 3}`.
3. Similarly, for `solutionVec2`, the constructor stores: `{1: 3, 3: 4}`.
4. Call `dotProduct` method on `solutionVec1` and pass `solutionVec2` as an argument.
5. The method iterates through the non-zero elements of `solutionVec2`.
6. For each non-zero element in `solutionVec2`, it checks if `solutionVec1` has a non-zero element at the same index.
7. It finds a match at index 3 (value 2 in `solutionVec1` and value 4 in `solutionVec2`). So it calculates `2 * 4 = 8`.
8. There are no other matching indices with non-zero values.
9. The method returns `8` as the final dot product.

Code



```java
import java.util.HashMap;
import java.util.Map;

public class Solution {
    private Map<Integer, Integer> nonZeroValues;

    // Constructor to initialize the non-zero values of the vector
    public Solution(int[] nums) {
        nonZeroValues = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                // Store only non-zero elements with their index
                nonZeroValues.put(i, nums[i]);
            }
        }
    }

    // Method to compute the dot product of two sparse vectors
    public int dotProduct(Solution vec) {
        int result = 0;
        for (Map.Entry<Integer, Integer> entry : vec.nonZeroValues.entrySet()) {
```

You must run your code first

Complexity Analysis

Time Complexity

1. **Constructor** (`Solution()`): `O(n)`, where `n` is the length of the input vector, as it iterates through all elements once.

2. **Dot Product Calculation** (`dotProduct`): `O(k)`, where `k` is the number of non-zero elements in the smaller of the two vectors, since lookup in the hash map is generally `O(1)`.

Overall Time Complexity: `O(n + k)`, where `k < n`. So, overall time complexity is `o(n)`.

Space Complexity

The space complexity of the algorithm is `O(k)`, where `k` is the number of non-zero elements in the vector. This is for the hash map or dictionary storing these elements.

Another Approach by Traversing Vector

The linear approach to computing the dot product of two sparse vectors relies on a straightforward yet effective method. Unlike strategies that involve storing and manipulating only the non-zero elements of the vectors, this approach retains and utilizes the entire structure of both vectors. The core logic behind this approach is the direct multiplication of corresponding elements from both vectors. Essentially, it mimics the standard process of dot product calculation, but it is specifically tailored for sparse vectors where many elements are zero.

This method is characterized by its simplicity and directness. By iterating through both vectors in parallel, we multiply each pair of corresponding elements and accumulate their products. The effectiveness of this approach is evident in cases where the vectors are not extremely sparse, or the distribution of non-zero elements is such that they align frequently. While it might seem less efficient due to the involvement of zero elements in the computations, its linear nature and the absence of complex data structures or lookups make it quite straightforward to implement and understand. This approach is a robust solution that aligns well with conventional vector operations, ensuring clarity and maintainability in practical applications.

Step-by-Step Algorithm

1. **Implement the Constructor** (`Solution()`):
   - The constructor accepts an array of integers as input.
   - Store the entire input array in the class member array.

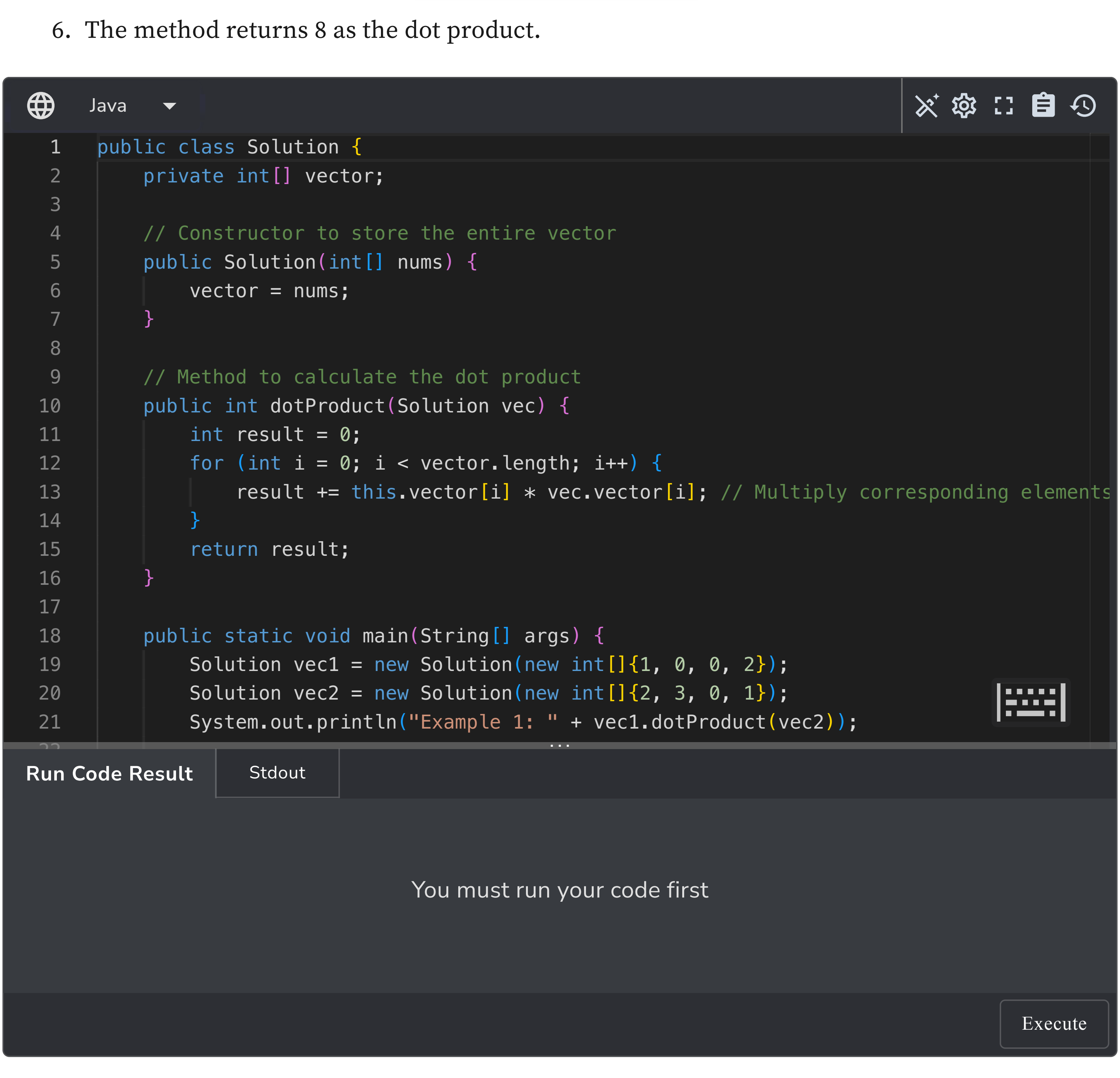
2. **Implement the** `dotProduct` **Method**:
   - This method accepts another `Solution` instance as input.
   - Initialize a variable, say `result`, to 0 for accumulating the dot product.
   - Iterate over the elements of both vectors simultaneously.
   - Multiply corresponding elements from both vectors and add the product to `result`.
   - Return `result` as the final dot product.

Algorithm Walkthrough

- **Example Input**:
  - `vec1 = [1, 0, 0, 2, 3]`

  - `vec2 = [0, 3, 0, 4, 0]`

1. Instantiate two `Solution` objects with `vec1` and `vec2`.
2. In each constructor, the entire vector is stored.
3. Call `dotProduct` on one object, passing the other as an argument.
4. The method iterates over the vectors: `1*0 + 0*3 + 0*0 + 2*4 + 3*0`.
5. The sum of these products is `0 + 0 + 0 + 8 + 0 = 8`.
6. The method returns 8 as the dot product.



```java
public class Solution {
    private int[] vector;

    // Constructor to store the entire vector
    public Solution(int[] nums) {
        vector = nums;
    }

    // Method to calculate the dot product
    public int dotProduct(Solution vec) {
        int result = 0;
        for (int i = 0; i < vector.length; i++) {
            result += this.vector[i] * vec.vector[i]; // Multiply corresponding elements
        }
        return result;
    }

    public static void main(String[] args) {
        Solution vec1 = new Solution(new int[]{1, 0, 0, 2});
        Solution vec2 = new Solution(new int[]{2, 3, 0, 1});
        System.out.println("Example 1: " + vec1.dotProduct(vec2));
```

You must run your code first

Complexity Analysis

Time Complexity

- **Constructor** (`Solution()`): `O(n)`, where `n` is the length of the input vector, for storing the entire vector.

- **Dot Product Calculation** (`dotProduct`): `O(n)`, as it linearly traverses the entire length of both vectors once.

Overall Time Complexity: `O(n)`, dominated by the linear traversal in the `dotProduct` method.

Space Complexity

The space complexity of the algorithm is `O(n)` as we store the entire vector in the constructor.

This linear approach is less efficient for sparse vectors with many zeros but is simpler and more straightforward.

← Previous
Dot Product Of Two Sparse Vectors (Medium)

Next →
Max Consecutive Ones III (Medium)

☐ Mark as Completed